SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

CHENNAI - 600089

DEPARTMENT OF PHYSICS - 2026 MINI PROJECT

NAME & REG. NO:

1.	**RA2611026020016**	**JACOB X**
2.	**RA2611026020017**	**KAVINESH V S**
3.	**RA2611026020018**	**VIGASH N S**

DEGREE: **1st YEAR - B TECH CSE AIML**

PROJECT TITLE: **LightLab: Room Illumination Simulator with Dark-Zone Detection and Automatic Light Placement**

MARK'S SPLIT UP

NOVELTY IN THE PROJECT (2 MARKS)	
DESIGN OF THE PROJECT (4 MARKS)	
WORKING CONCEPT OF THE PROJECT (2 MARKS)	
CONTRIBUTION TO THE PROJECT (2 MARKS)	
TOTAL (10 MARKS)	10

SIGNATURE OF THE GUIDE

SIGNATURE OF THE HOD

Title:

LightLab : Room Illumination Simulator with Dark-Zone Detection and Genetic-Algorithm Light Placement

Abstract

Application Link: https://lightlab1.streamlit.app

GitHub Link: https://github.com/nsvigash1-netizen/lightlab

LightLab 2.0 is a Python desktop application designed to demonstrate how photometric principles can be used to analyse and improve the placement of indoor light sources. The user defines the room dimensions, enters the position, mounting height and luminous flux of each light, and selects a minimum illuminance threshold. The program divides the floor into a numerical grid and estimates illuminance at every grid point. Each source is modelled as an isotropic point source. Its luminous intensity is approximated as luminous flux divided by 4π, and horizontal-plane illuminance is calculated using the inverse-square law and the cosine of the incidence angle. Contributions from multiple sources are added using superposition. A colour heatmap displays the predicted distribution, while the application reports minimum, maximum and average illuminance, a minimum-to-average uniformity index, and the estimated area and percentage below the selected threshold. A module uses a simple genetic algorithm to search for light positions. Candidate arrangements are scored using illumination deficit, dark-zone fraction and uniformity; selection, crossover, mutation and elitism are applied over successive generations. The best arrangement found is displayed in the same simulator and compared with the previous arrangement when available. The project combines physics, numerical computation, graphical visualisation and an introductory optimization method in one application. It is intended for learning and preliminary comparison, not certified lighting design. The model neglects wall and ceiling reflections, obstacles, shadows, fixture beam patterns and room surface properties. Therefore, results are theoretical estimates and should be verified with real measurements before practical use.

Key words:

Room illumination; illuminance; luminous flux; inverse-square law; photometry; dark-zone detection; numerical simulation; genetic algorithm; Python.

Introduction of the Project:

Lighting affects the visibility and usability of indoor spaces. A room may have a high average light level while some corners or work areas remain comparatively dim. Placing more lamps without analysing their positions can also waste energy or produce an uneven distribution. LightLab is a small software tool that demonstrates how a physics model can be used to estimate illumination across a room and compare alternative lamp arrangements.

The application represents the floor as a two-dimensional grid. A user enters the room dimensions and the properties of one or more light sources. The program estimates illuminance at each grid point, presents the values as a colour heatmap, identifies points below a user-selected threshold, and calculates summary measurements. Its automatic-placement module then uses a genetic algorithm to search for a better arrangement of a fixed number of identical lights.

The project is designed as an educational demonstration of photometry, numerical modelling and optimization. It does not replace a professional lighting simulation package or a physical lux-meter survey.

Aim:

To develop a Python application that estimates the distribution of illuminance on a room floor and helps users examine and improve the placement of light sources.

Specific objectives:

- Calculate floor illuminance using a simplified point-source model based on the inverse-square law.
- Display the predicted illumination as a colour-coded heatmap.
- Detect floor regions below a user-defined lux threshold and estimate their area.
- Apply a genetic algorithm to search for light positions that reduce the calculated illumination deficit and improve uniformity.

Apparatus required:

1. Laptop or desktop computer running Windows, Linux or macOS.
2. Python 3 installation.
3. Visual Studio Code, IDLE or another Python editor (optional but useful).
4. NumPy library for numerical calculations.
5. Matplotlib library for plotting the illumination heatmap.
6. Tkinter GUI library for the desktop interface (included with many standard Python installations; on some Linux systems it may need to be installed separately).
7. LightLab Python source file (lightlab.py). No physical lamp, sensor or measuring instrument is required to run the simulation.

Procedure:

1. Install Python 3 and the required packages by running: python -m pip install numpy matplotlib.
2. Save the application source code as lightlab.py and run it using: python lightlab.py.
3. Enter the room width and length in metres and choose the minimum desired illuminance in lux.
4. To add a light manually, enter its floor-plan coordinates (x and y), mounting height above the floor, and luminous flux in lumens; then select Add Light.
5. Add more light sources as required. The installed-light table displays the coordinates, height and luminous flux of each source.
6. Select Generate Illumination Map. The program creates a 70 × 70 grid of floor points, calculates the contribution from each light and adds the contributions at every point.
7. Study the heatmap and the displayed minimum, maximum and average illuminance, uniformity index, dark-zone area and dark-zone percentage.
8. For automatic placement, enter the required number of lights, their common mounting height and their common luminous flux in the optimizer panel. The current room dimensions and target lux are used by the search.
9. Select Optimize Light Placement. The genetic algorithm generates candidate arrangements, evaluates their scores, selects parent arrangements, performs crossover and mutation, and retains the best candidates over successive generations.
10. After optimization, inspect the new heatmap and the before-and-after dark-zone and uniformity figures. Repeat with other room sizes, thresholds or light specifications to compare cases.

Concept:

1. Luminous flux and luminous intensity

Luminous flux (Φ) is the total visible-light output of a source and is measured in lumens (lm). For an ideal isotropic point source, the flux is distributed uniformly over a sphere. The luminous intensity is therefore approximated by $I = \Phi / (4\pi)$, where I is measured in candela (cd).

2. Distance and illuminance on a horizontal floor

Let a light be at horizontal coordinates (xL, yL) and height h above the floor. For a floor point (x, y), the straight-line distance is $r = \sqrt{[(x - xL)^2 + (y - yL)^2 + h^2]}$. The cosine of the incidence angle for a horizontal floor is h/r. Using $E = I \cos\theta / r^2$ gives $E(x,y) = \Phi h/(4\pi r^3)$. Illuminance E is measured in lux (lm/m^2).

3. Superposition of several lights

The program calculates the illuminance due to each light independently and adds the contributions: $Etotal(x,y) = \Sigma\, Ei(x,y)$. This assumes that the sources act independently and that their illuminance contributions can be added.

4. Dark-zone detection and uniformity

A grid point is marked as a dark-zone point when its estimated illuminance is lower than the user-selected threshold Et. The dark-zone percentage is the number of flagged grid points divided by the total number of grid points, multiplied by 100. The dark-zone area is estimated by multiplying this fraction by the room floor area. The application also calculates a simple uniformity index $U0 = Emin/Eavg \times 100\%$, where Emin and Eavg are the minimum and average grid illuminance values. A higher index indicates a more even distribution in this simplified model.

5. Genetic algorithm for automatic placement

Each candidate solution is an arrangement of the specified number of lights, with each light represented by an (x, y) position. The program evaluates the illumination produced by each candidate. A lower fitness score is preferred: $S = 0.55D + 0.25Z + 0.20(1 - U)$, where D is the average normalized illumination deficit below the target, Z is the fraction of grid points below the target, and $U = Emin/Eavg$ is the normalized minimum-to-average uniformity ratio. The algorithm uses tournament selection, uniform crossover, random mutation and elitism (preserving the two best candidates). Positions are kept within the room boundary.

The algorithm searches for a good arrangement; it does not prove that the global best arrangement has been found. In the current program, the number of lights, their height and their luminous flux are fixed during a search.

Diagram:

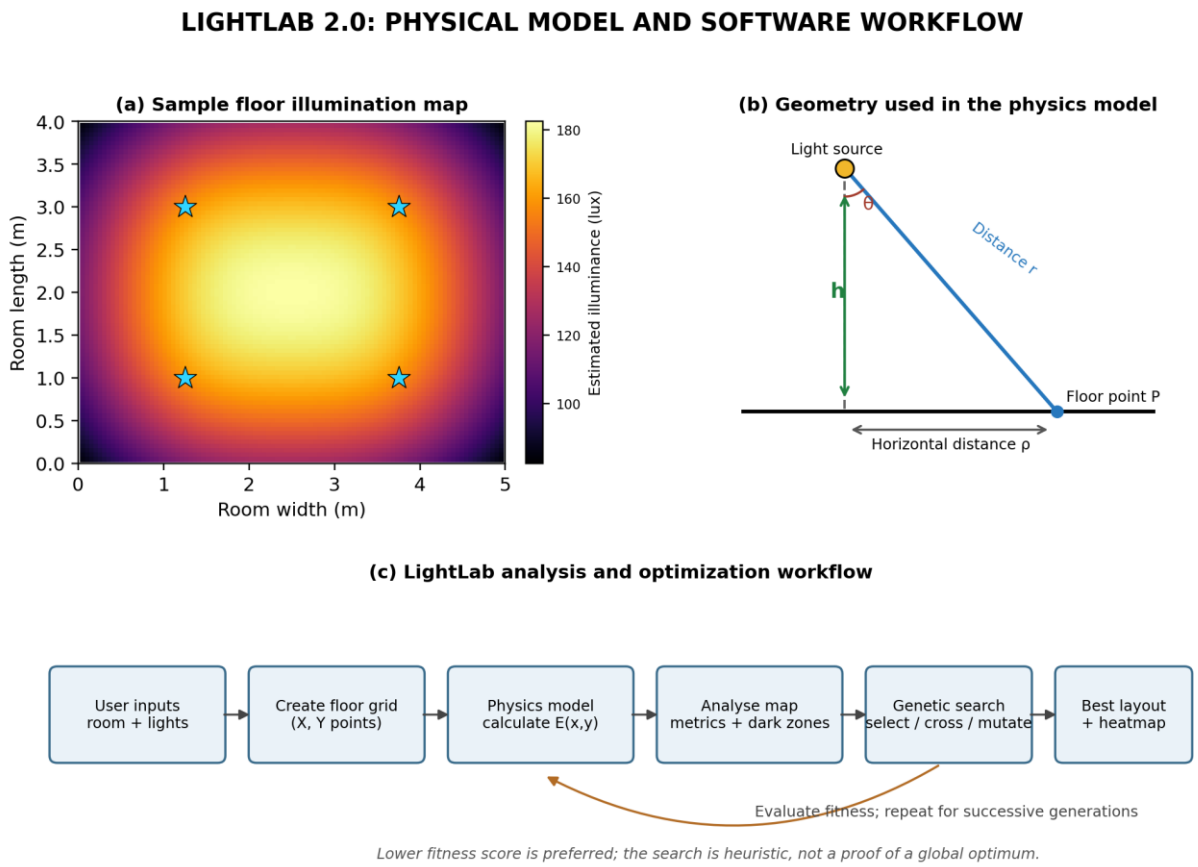


Figure 1. Conceptual floor-illumination map, point-source geometry and LightLab analysis/optimization workflow. The floor map is an illustrative calculation for a 5 m × 4 m room with four identical modelled sources; it is not a measurement of a real room.

Applications:

1. Educational demonstration of the inverse-square law, luminous intensity, illuminance and superposition.
2. Preliminary comparison of possible ceiling-light positions in a small room, classroom, study area or laboratory model.
3. Visual identification of regions that fall below a user-selected illumination threshold in the simulation.
4. Demonstration of how numerical modelling and genetic algorithms can be applied to a simple engineering layout problem.
5. A starting point for student experiments comparing different room dimensions, number of lights, mounting heights and luminous flux values.

Additional Information:

Implementation details

The graphical interface is created with Tkinter. NumPy is used to construct the floor grid and perform the calculations, while Matplotlib displays the heatmap inside the application window. The final heatmap uses a 70 × 70 grid. To reduce computation during the search, the optimizer evaluates candidates on a 40 × 40 grid.

The genetic algorithm in the current code uses a population of 36 candidate arrangements and 45 generations. It preserves the best two candidates in each generation. Tournament selection compares three randomly chosen candidates; uniform crossover combines coordinates from two parents; and Gaussian mutations alter selected coordinates. The mutation scale gradually decreases during the search. The number of lights is limited to 1–12 in the interface.

Suggested demonstration case

A demonstration can use a 5 m × 4 m room, a target threshold of 100 lux, four lights, a mounting height of 2.5 m and 6000 lm per light. These are example input values for demonstrating the software, not a universal lighting recommendation. The student can compare the manually entered arrangement with the arrangement returned by the optimizer.

Assumptions and limitations

The model treats each lamp as an isotropic point source and assumes a flat horizontal floor. It does not include reflections from walls, ceiling or floor; furniture and other obstructions; shadows; non-uniform beam patterns; lamp ageing; or changes in surface reflectance. The area calculation is an approximation based on the finite grid. The genetic algorithm is stochastic and may return different arrangements on different runs. The current optimizer also does not impose a minimum separation between lights or search for different heights or light outputs.

For these reasons, the application should be presented as an educational simulation and preliminary layout comparison tool. A real installation should be checked with a calibrated lux meter and, where needed, a professional lighting-design method.

Conclusion:

LightLab demonstrates how a physics-based mathematical model can be converted into a practical desktop application. By applying the inverse-square law and the cosine dependence of illumination on the receiving surface, it estimates the distribution of illuminance over a room floor. Its heatmap and dark-zone analysis make variations in the predicted lighting easier to understand.

The automatic-placement module extends the project from simulation to optimization. A genetic algorithm compares candidate positions using illumination deficit, dark-zone fraction and uniformity, then displays the best arrangement found during its search. The project therefore combines photometry, numerical computation, data visualization and an introductory evolutionary optimization method.

Although its simplified assumptions limit direct real-world accuracy, it provides a useful platform for learning, experimentation and future improvement.